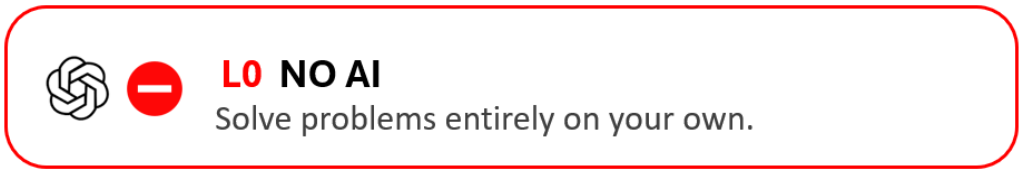
W1 PRACTICE

PART 1 – EXPLORATION

*What do you need to **be able to do** for next week?*

- Identify the **type** of variable
- Manipulate **nullable** and **non-nullable** variables.
- Differentiate between **final** and **const**.
- Manipulate **strings, lists, and maps.**
- Use **loops** and **conditions** to control flow.
- Define and call functions with positional and **named arguments**, understand **arrow syntax**



How to run Dart code?

You can write you code on VS Code, or using this online editor

- Install Dart and Flutter SDK
- Online Dart Compiler

Resources for this research

To help you complete this handout, use the following resources:

- ✓ Variables
- ✓ Null Safety
- ✓ Built-in types
- ✓ Lists
- ✓ Loops
- ✓ Conditions
- ✓ Functions

EX 1 - Type Inference

Built-in types

EXPLAIN : Explain how Dart infers the type of a variable.

Dart infers the type of a variable by reading the variable's references. It identify the type by the assigned value when the variable is initialized.

CODE : Complete the bellow code to illustrate the concepts:

```
// Declare a int variable using var or final and let Dart infer its type
var variable = 100;
  print("This variable has type of (${variable.runtimeType}) and has value
of: ${variable}");

// Define a variable with an explicit String type
String stringType = 'what is dart';
  print("This variable has type of (${stringType.runtimeType}) and has value
of: ${stringType}");
```

EX 2 - Nullable and Non-Nullable Variables

Variables Null Safety

EXPLAIN : Explain nullable vs non-nullable variables.

Nullable variable is a variable that can be initialized and contain either a value and null while Non-nullable variable is a variable that is initialized and must be a value consisted inside the variable and cannot be null.

Non-nullable variable also comes in default for each variable unless there is an "?" indicator to specify if the variable is nullable or non-nullable.

EXPLAIN : When is it useful to have nullable variables?

It's really useful to be able to represent the *absence* of a value.

CODE : Complete the bellow code to illustrate the concepts:

```
// Declare a nullable integer variable and assign it a null value
  int? num1 = null;
  print(num1.runtimeType);

// Declare a non-nullable integer variable and assign it a value
   int num2 = 10;
   print(num2);

// Assign a new value to the nullable variable
```

```
   num1 = 20;
   print(num1.runtimeType);
```

EX 3 - Final and const

EXPLAIN : Describe the difference between final and const.

A final object cannot be modified, its fields can be changed and It is mutable. In comparison, a const object and its fields cannot be changed and it is inmutable.

CODE : Complete the bellow code to illustrate the concepts:

```
// Declare a final variable and assign it the current date and time
   final now = DateTime.now();

// Can you declare this variable as const? Why?
   You can't declare "now" variable as const because DateTime continuously
change every second and the value of current date time is unknown during
compile time which violient const principle.

// Declare a const variable with a integer value
   const val = 10;

// Can you reassign the value of this final variable? Why?
   You can't reassign the value of "val" because you cant reassigned the
const value
```

EX 4 - Strings, Lists and Maps

Lists Built-in types

CODE : Complete the bellow code to illustrate the concepts:

```
Strings:
// Declare two strings: firstName and lastName and an integer:age
   String firstName;
   String lastName;
   int age;

// Concatenate the 2 strings and the age
   String concat = firstName + lastName + age.toString();

// Print result
   print(concat);

Lists:
// Create a list of integers
  List<int> nums = [2, 3, 4, 5];
```

```
// Add a number to the list
  nums.add(3);

// Remove a number from the list
  nums.remove(3);

// Insert a number at a specific index in the list
  nums.insert(2, 20);

// Iterate over the list and print each number
  for (int i in nums) {
    print(i);
  }
```

Maps:

```
// Create a map with String keys and integer values
  var maps = {
    'name': 'tember',
    'val': 10
  };

  // Add a new key-value pair to the map
  maps['young'] = false;

  // Remove a key-value pair from the map
  maps.remove('name');

  // Iterate over the map and print each key-value pair
  for(var i in maps.values) {
    print(i);
  }
```

EXPLAIN : When should I use a Map instead of a List?

Map is used when the data is stored with key-value and accessing value based on keys. List is used when the data is in order and accessing its value based on index

EXPLAIN : When should I use a Set instead of a List?

Set is needed when collection of unique values and do not want duplicates. List is used when duplicates value are allowed and considering of order.

EX 5 - Loops and Conditions

Loops Conditions

CODE : Complete the bellow code to illustrate the concepts:

```
// Use a for-loop to print numbers from 1 to 5
   for(int i = 0; i < 5; i++) {
    print(i+1);
  }

// Use a while-loop to print numbers while a condition is true
   int j = 0;
   while(true){
     print(j);
     j++;
   }

// Use an if-else statement to check if a number is even or odd
  int number = 10;
  if(number%2 == 0) {
    print("Num is even");
  } else {
    print("Num is odd");
  }
```

EX 6 - Functions

Functions

EXPLAIN : Compare positional and named function arguments

Positional argument wrapped by [] and its value is called and set to the default value when using the function.

Named argument wrapped by {} and must use the argument name when using the function

EXPLAIN : Explain when and how to use arrow syntax for functions?

Arrow syntax is used for functions that contain a single statement. It is useful when passing anonymous functions as arguments.

CODE : Complete the bellow code to illustrate the concepts:

Defining and Invoking a Function:

```
// Define a function that takes two integers and returns their sum
   int sum(int a, int b) {
      return a + b;
   }


// Call the function and print the result
   int res = sum(2, 9);
   print(res);
```

Positional vs Named Arguments:

```
// Define a function that uses positional arguments
   void createPosition([int x = 0 , int y = 0]) {
      print('x = $x, y = $y');
   }

// Define another function that uses named arguments with the required
keyword (ex: getArea with rectangle arguments)
   void rectangle({int getArea = 20}) {
      print('area: $getArea');
   }

// Call both functions with appropriate arguments
   createPosition();
   rectangle();
```

EXPLAIN : Can positional argument be omitted? Show an example

Yes, the positional argument can be omitted; the value will be assigned as the default value.

```
void createPosition([int x = 0 , int y = 0]) {
      print('x = $x, y = $y');
}
createPosition();
createPosition(10);
```

EXPLAIN : Can named argument be omitted? Show an example

Yes, the named argument can be omitted; the value will also be assign as the default value.

```
void rectangle({int getArea = 200}) {
```

```
      print('area: $getArea');
}
rectangle();
rectangle(400);
```

CODE : Complete the bellow code to illustrate the concepts:

Arrow Syntax:

```
// Define a function using arrow syntax that squares a number
int squareNum(int num) => (num*num);

// Call the arrow function and print the result
print(squareNum(20));
```